

IMPLEMENTATION & MIGRATION RUNBOOK

KPTC Scheduler

独立Linuxサーバー構築・移行手順書

内部スケジューラーと外部空き状況を別サーバーへ分離する本番構成

対象	内部Linuxサーバー / 外部Linuxサーバー
連携	内部から外部への認証付きHTTPS JSON送信
基準	GitHub main / コミット abe28ac / 2026年8月15日

一 目的・前提・適用範囲

現行のさくら同居構成から、内部公開と外部公開を物理的または論理的に分離したLinuxサーバー2台構成へ移行します。

内部 スケジューラー、PHP API、SQLite、操作履歴。社内LANまたはVPNだけから利用。	外部 3試験室の空き状況、受信API、3か月JSON。インターネット公開。	連携 内部から外部へのHTTPS送信だけを許可。外部から内部へ接続しない。
---	---	---

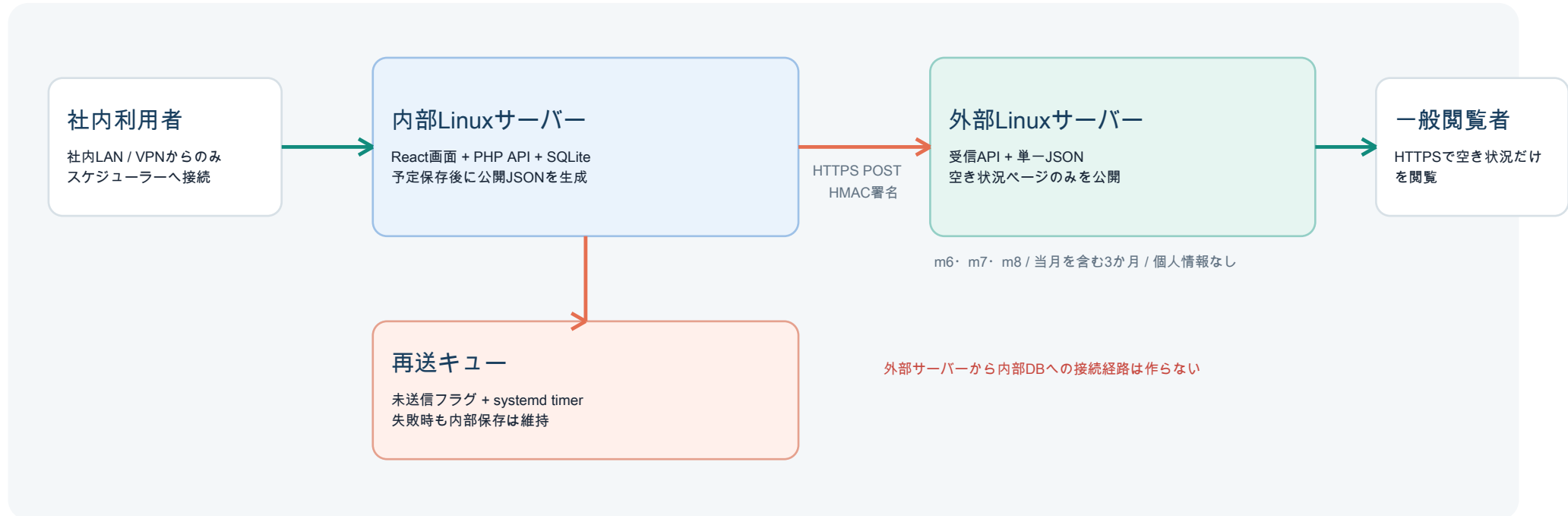
項目	本手順書の前提	確定が必要な事項
OS	systemdを利用できるLinux。例はUbuntu / Debian系を基準	採用ディストリビューションと保守期限
Web	Nginx + PHP-FPM 8.2以上 + SQLite拡張 + cURL拡張	組織標準のWebサーバーがApacheの場合は設定を読み替え
ビルド	Node.js 22.13以上、pnpm。原則としてCIまたは管理端末で実行	本番サーバーへNode.jsを置くか否か
DNS	内部FQDNは内部DNS、外部FQDNは公開DNS	正式FQDN、証明書発行元、内部CIDR
認証	公開連携はHTTPS + HMAC-SHA256。内部画面は正式認証へ変更	SSO / LDAP / OIDC等の方式と権限区分

重要な本番化条件

- 現行の利用者選択式デモログインは本人性を保証しません。内部ネットワーク制限だけで本番運用せず、正式認証を実装します。
- 外部サーバーにはSQLite、内部API、利用者名、予定件名、メモ、操作履歴を配置しません。
- 以下の<INTERNAL_FQDN>等はそのまま実行せず、環境固有値へ置換してレビューします。

二 目標アーキテクチャ

公開側DBは設置せず、外部サーバーは当月を含む3か月分の単一JSONだけを保持します。



通信	向き	許可	目的
社内ブラウザ → 内部	HTTPS 443	社内CIDR / VPNのみ	予定・ マスタ・ 履歴の利用
内部 → 外部受信API	HTTPS 443	HMAC必須。可能なら送信元IPも制限	公開JSONの上書き送信
一般ブラウザ → 外部	HTTPS 443	全世界または公開対象地域	空き状況の閲覧
外部 → 内部	なし	FWで拒否	経路を作成しない
管理者 → 両サーバー	SSH 22等	管理ネットワーク / 踏み台のみ	保守・ 配備・ 復旧

三 現行実装との差分

コミットabe28acは同一サーバー上のローカルファイル置換です。2サーバー化には次の変更が必要です。

対象	現行	2サーバー化後	実装判定
公開JSON出力	availability-json.phpがローカルへrename	内部でJSON生成後、外部受信APIへPOST	必須変更
公開JSON読出し	同じPHPが同じ保存先を参照	外部サーバーの専用保存先だけを参照	設定変更
内部DBパス	api.php内で相対パス固定	KPTC_INTERNAL_SCHEDULER_DB環境変数を優先	必須変更
配布物	dist-sakuraに内部・外部画面とPHPを同梱	内部用と外部用をホワイトリストで別パッケージ化	必須変更
再送	未送信フラグ、次回APIアクセスで再試行	加えて5分間隔timerで能動再送	必須変更
認証	利用者選択式デモ認証	組織認証 + セッション。公開送信はHMAC	本番前必須
公開受信	なし	署名・時刻・JSON構造・世代を検証して原子的置換	新規実装

推奨ファイル分割

区分	推奨ファイル	役割
共通モデル	server/common/availability-model.php	予定から公開可能なJSONを生成。ファイルI/OやHTTPを持たない
内部送信	server/internal/availability-publisher.php	HMAC署名し、外部APIへHTTPS POST
内部CLI	server/internal/publish-availability-cli.php	SQLite読込、生成、送信、終了コード返却
外部受信	server/public/receive-availability.php	署名・スキーマ・世代検証、単一JSONを原子的置換
外部読出し	server/public/public-availability.php	保存済みJSONだけをGETで返す
配備	scripts/package-internal.sh / package-public.sh	明示リストから各配布物を生成

四 公開JSONと送信プロトコル

ブラウザ向け公開データとサーバー間認証を分離し、リプレイ・改ざん・古いデータの上書きを防ぎます。

項目	仕様
URL	POST https://<PUBLIC_FQDN>/api/v1/availability
Content-Type	application/json; charset=utf-8
X-KPTC-Timestamp	送信時UNIX秒。受信時刻との差が300秒以内
X-KPTC-Signature	hex(HMAC-SHA256, timestamp + LF + rawBody, sharedSecret))
本文	schemaVersion, sourceVersion, updatedAt, rangeStart, rangeEnd, availability
室ID	m6 / m7 / m8以外を拒否
状態	maintenance / reserved / morning_available / afternoon_availableのみ
期間	rangeStartは当月1日、rangeEndは3か月目末日。最大93日程度
世代	新rangeStartを優先。同一rangeStartではsourceVersionが古い送信を409で拒否
サイズ	受信上限128 KiB。未知キー、個人情報らしい項目、日付形式不正を拒否
保存	同一ディレクトリの一時ファイルへ0600で書き、fsync相当後にrename
応答	200=更新、202=同一世代、400=形式不正、401=署名不正、409=旧世代、503=保存失敗

秘密鍵の作成例

```
openssl rand -hex 32
# 出力をパスワード管理庫へ登録し、内部送信側と外部受信側だけへ配布する
```

- 共有鍵はGit、PDF、配備物、ログへ記録しません。/etc/kptc/*.envはroot所有・0640とします。
- NTP / chronyで両サーバーの時刻を同期します。時刻ずれは署名検証失敗の原因になります。
- 可能なら外部FWまたはNginxで内部サーバーの固定送信元IPを追加制限します。

五 コード変更の実装手順

最初に送受信処理を分離し、ローカル同居動作と遠隔送信を環境変数で切り替えられるようにします。

順序	変更	完了条件
1	kptc_build_public_availability()を純粋な生成関数として残す	同じ入力state/versionから同じ公開構造を生成
2	kptc_publish_availability()のファイル書込を送信インターフェースへ分離	local / httpsの実装を設定で選択
3	PHP cURLでPOSTし、timestampとrawBodyからHMACを生成	2xxのみ成功。接続・TLS・4xx/5xxは例外
4	受信APIを新設し、rawBodyのままHMAC検証後にJSON解析	署名比較はhash_equals()を使用
5	public/api.phpのdb()でKPTC_INTERNAL_SCHEDULER_DBを優先	Web APIとCLIが同じSQLiteを参照
6	保存・削除・undo後は内部DBコミットを先行し、連携失敗時はpending=1	外部障害でも内部予定が失われない
7	CLIは送信成功0、再試行可能失敗1、設定不正2を返す	systemdが失敗を検知可能
8	送信JSONに利用者名・件名・メモ・履歴がないことをテスト	許可キーのホワイトリスト試験が成功

内部側環境変数

```
KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite
KPTC_PUBLIC_AVAILABILITY_MODE=https
KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://<PUBLIC_FQDN>/api/v1/availability
KPTC_PUBLIC_AVAILABILITY_SECRET=<64_HEX_SECRET>
KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10
```

外部側環境変数

```
KPTC_PUBLIC_AVAILABILITY_JSON=/var/lib/kptc-public/public-availability.json
KPTC_PUBLIC_AVAILABILITY_SECRET=<64_HEX_SECRET>
KPTC_PUBLIC_AVAILABILITY_MAX_SKEW=300
```

六 内部・外部配布物の分離

同じdist-sakuraを両方へコピーしてはいけません。配備対象をホワイトリストで固定します。

内部パッケージに含める	外部パッケージに含める	外部へ含めない
index.html / 内部画面assets	reservations.html / 公開画面assets	index.html / 内部画面assets
api.php / 内部送信処理	public-availability.php	api.php / SQLite処理
公開モデル / 内部CLI	receive-availability.php / 読出し補助	内部CLI / 初期データ / 操作履歴
ロゴ等の共通画像	ロゴ等の必要画像のみ	不要なソースマップ・開発設定

ビルドと検査

```
corepack enable
pnpm install --frozen-lockfile
pnpm run check
pnpm test
pnpm run build

# 推奨: package-internal と package-public を別ディレクトリへ生成
./scripts/package-internal.sh dist-internal
./scripts/package-public.sh dist-public
```

検査	内部	外部
必須ファイル	index.html, api.php, scheduler assets, sender CLI	reservations.html, receiver, reader, public assets
禁止ファイル	公開受信secretを含むファイル	api.php, group-watcher.sqlite, 内部JS, DB移行コード
機密検索	実鍵・本番パス・個人情報が成果物にない	同左。さらにmembers / schedules / audit_logsがない
ハッシュ	配布tar.gzのSHA-256を記録	配布tar.gzのSHA-256を記録

Viteは現在2入口を一括生成します。実装時は2つのVite設定へ分けるか、生成後の配布スクリプトで必ず明示コピーしてください。除外リスト方式より、含めるファイルを列挙する方式が安全です。

七 Linuxサーバー共通準備

以下はUbuntu / Debian系の例です。RHEL系ではdnf、nginx、php-fpm、php-cli、php-pdo、php-sqlite3、php-curl等へ読み替えます。

パッケージ導入例

```
sudo apt update
sudo apt install -y nginx php-fpm php-cli php-sqlite3 php-curl sqlite3 curl ca-certificates chrony
sudo systemctl enable --now nginx chrony
```

サービスアカウントとディレクトリ

```
sudo useradd --system --home /nonexistent --shell /usr/sbin/nologin kptc
sudo install -d -o root -g root -m 0755 /opt/kptc/releases
sudo install -d -o root -g root -m 0755 /etc/kptc
sudo install -d -o kptc -g kptc -m 0700 /var/lib/kptc-scheduler
sudo install -d -o kptc -g kptc -m 0700 /var/lib/kptc-public
sudo install -d -o root -g root -m 0755 /var/www/kptc-internal
sudo install -d -o root -g root -m 0755 /var/www/kptc-public
```

設定	推奨値
時刻	Asia/Tokyo。chrony同期を有効化し、timedatectl / chronyc trackingで確認
更新	OSセキュリティ更新の定期適用。PHPとNginxはサポート版を固定
SSH	鍵認証、root直接ログイン禁止、管理CIDRまたは踏み台だけ許可
SELinux	RHEL系は無効化せず、Web読取・ネットワーク送信・データ書込に必要なコンテキストを付与
AppArmor	Ubuntu系でプロファイルを使う場合、JSON保存先と外向きHTTPSを許可
ログ	journal + Nginx access/error。秘密鍵・JSON本文・予定情報を出力しない

PHP-FPM実行ユーザーとkptc所有ファイルの関係は、専用poolを作るのが最も明確です。既定www-dataを使う場合は、必要最小限のグループ権限で調整します。

八 内部サーバーの構築

内部WebルートとSQLiteを分離し、社内ネットワーク以外からの到達をFWとNginxの両方で拒否します。

配置	推奨パス	所有・権限
リリース	/opt/kptc/releases/<VERSION>/internal	root:root / 0755、ファイル0644
公開リンク	/var/www/kptc-internal/current	上記リリースへのsymlink
SQLite	/var/lib/kptc-scheduler/group-watcher.sqlite	kptc:kptc / 0600
環境設定	/etc/kptc/internal.env	root:kptc / 0640
バックアップ	/var/backups/kptc-scheduler	root:root / 0700、別媒体へ転送

初回配備例

```
VERSION=2026.08.15-1
sudo install -d -o root -g root -m 0755 /opt/kptc/releases/$VERSION/internal
sudo cp -a dist-internal/ /opt/kptc/releases/$VERSION/internal/
sudo ln -sfn /opt/kptc/releases/$VERSION/internal /var/www/kptc-internal/current
sudo chown -R root:root /opt/kptc/releases/$VERSION
sudo find /opt/kptc/releases/$VERSION -type d -exec chmod 0755 {} \;
sudo find /opt/kptc/releases/$VERSION -type f -exec chmod 0644 {} \;
```

既存SQLiteの移行

```
# さくら側でアプリ停止または書込停止後にSQLite整合バックアップを取得
sqlite3 /home/apfelrunner/GW/group-watcher.sqlite \
".backup '/tmp/group-watcher.sqlite'"
sha256sum /tmp/group-watcher.sqlite

# 暗号化された管理経路で内部サーバーへ転送後
sudo install -o kptc -g kptc -m 0600 group-watcher.sqlite \
/var/lib/kptc-scheduler/group-watcher.sqlite
sqlite3 /var/lib/kptc-scheduler/group-watcher.sqlite 'PRAGMA integrity_check;'
```

- DBファイルだけでなくWAL/SHMが存在するため、稼働中の単純cpは禁止します。.backupまたは停止後コピーを使います。
- 初回アクセス前に旧DBのSHA-256、件数、最新versionを記録します。

九 外部サーバーの構築

公開サーバーには空き状況画面、受信API、読み出しAPI、単一JSONだけを配置します。

配置	推奨パス	所有・権限
リリース	/opt/kptc/releases/<VERSION>/public	root:root / 0755、ファイル0644
公開リンク	/var/www/kptc-public/current	上記リリースへのsymlink
単一JSON	/var/lib/kptc-public/public-availability.json	kptc:kptc / 0600
環境設定	/etc/kptc/public.env	root:kptc / 0640
TLS鍵	OS標準の証明書管理パス	rootのみ。自動更新後にNginx reload

初回配備と空JSON

```
VERSION=2026.08.15-1
sudo install -d -o root -g root -m 0755 /opt/kptc/releases/$VERSION/public
sudo cp -a dist-public/. /opt/kptc/releases/$VERSION/public/
sudo ln -sfn /opt/kptc/releases/$VERSION/public /var/www/kptc-public/current

# 受信前は公開APIが503を返す状態でよい。手作業で架空JSONを置かない。
curl -i https://<PUBLIC_FQDN>/public-availability.php
```

外部サーバーの禁止事項

禁止	確認方法
内部API・スケジューラー画面を置く	curl /api.php と /index.html が404または非公開であること
SQLiteやバックアップをWebルートへ置く	findとNginx設定をレビュー。拡張子denyだけに依存しない
受信APIを署名なしで受け付ける	署名なしPOSTが401であること
JSON本文をアクセスログへ記録する	ログ形式とアプリerror_logを確認
ディレクトリー覧を有効化する	autoindex off. 未知パスは404

+ Nginx・PHP-FPM・FW設定

内部と外部でserver blockを分け、実行可能なPHPをexact matchで限定します。

内部Nginxの要点

```
server {
    listen 443 ssl http2;
    server_name <INTERNAL_FQDN>;
    root /var/www/kptc-internal/current;
    allow <INTERNAL_CIDR>; deny all;
    location / { try_files $uri $uri/ /index.html; }
    location = /api.php { include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root/api.php;
        fastcgi_pass unix:/run/php/kptc-internal.sock; }
    location ~ \.php$ { return 404; }
}
```

外部Nginxの要点

```
server {
    listen 443 ssl http2;
    server_name <PUBLIC_FQDN>;
    root /var/www/kptc-public/current;
    client_max_body_size 128k; autoindex off;
    location / { try_files $uri $uri/ =404; }
    location = /public-availability.php { include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root/public-availability.php;
        fastcgi_pass unix:/run/php/kptc-public.sock; }
    location = /api/v1/availability { limit_except POST { deny all; }
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root/receive-availability.php;
        fastcgi_pass unix:/run/php/kptc-public.sock; }
    location ~ \.php$ { return 404; }
}
```

層	内部	外部
FW	443は社内CIDR/VPNのみ。SSHは管理CIDRのみ	443公開。SSHは管理CIDRのみ
PHP-FPM	専用pool、kptcユーザー、内部envを明示継承	専用pool、kptcユーザー、外部envを明示継承
TLS	内部CAまたは組織証明書。Secure Cookieが有効になる構成	公開CA証明書、自動更新、TLS1.2以上
ヘッダー	CSP、X-Content-Type-Options、frame-ancestors	同左。受信APIはCache-Control: no-store
レート制限	ログイン/APIに適用	受信APIに厳しめ、閲覧APIは通常キャッシュ60秒

十一 自動送信・再送・月替わり

予定保存直後の送信に加え、5分間隔の再送と月初の期間更新をsystemd timerで保証します。

service例

```
# /etc/systemd/system/kptc-availability-publish.service
[Unit]
Description=Publish KPTC lab availability
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
User=kptc
Group=kptc
EnvironmentFile=/etc/kptc/internal.env
ExecStart=/usr/bin/php /var/www/kptc-internal/current/publish-availability-cli.php
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ReadWritePaths=/var/lib/kptc-scheduler
```

timer例

```
# /etc/systemd/system/kptc-availability-publish.timer
[Unit]
Description=Retry KPTC availability publication

[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
Persistent=true
RandomizedDelaySec=30

[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now kptc-availability-publish.timer
systemctl list-timers kptc-availability-publish.timer
sudo systemctl start kptc-availability-publish.service
journalctl -u kptc-availability-publish.service -n 50 --no-pager
```

- CLIは毎回送っても受信側が同一世代を202として扱うため、冪等にします。
- 当月が変われば同じ予定versionでもrangeStartが新しくなるため、必ず受理される比較規則にします。
- 連続失敗回数、最後の成功時刻、外部JSONのupdatedAtを監視対象にします。

十二 移行・切替手順

外部を先に準備し、内部を最後に切り替えます。各段階で戻し先を残します。

順序	作業	判定 / 戻し
1	正式FQDN、CIDR、TLS、認証方式、保守責任者を確定	未確定があれば本番切替しない
2	開発環境で送信・受信・ビルド分離を実装し、自動試験	mainへ統合する前にレビュー
3	外部サーバーへpublic/パッケージを配備	空JSON時503、静的画面表示、内部資産なし
4	HMAC鍵を安全に両サーバーへ登録	権限0640、ログ・Gitに鍵なし
5	内部サーバーへinternal/パッケージを配備	新規DBで起動確認後、停止
6	さくら側を書込停止し、SQLite .backupとSHA-256を取得	旧環境は読取専用で保持
7	内部へDB移送、integrity_check、件数・version確認	不一致なら旧環境再開
8	内部API起動、正式認証、予定表示・保存・undo確認	公開送信前に内部機能を確認
9	手動publishを実行し、外部updatedAt/sourceVersionを確認	失敗時はpendingのまま原因修正
10	timer有効化、内部DNS・外部DNSを切替	DNS TTLを事前短縮
11	24～72時間監視後、さくら環境を停止。バックアップは保持	即時削除しない

切替時の書込停止

- ・移行開始時刻、最終予定version、最終操作履歴IDを記録します。
- ・旧環境と新環境への二重書込を禁止します。DNSだけでなく旧APIを書込停止にします。
- ・外部公開ページは切替中も最後に受信したJSONを表示できるため、内部停止の影響を分離できます。

十三 受入試験

機能だけでなく、公開情報の最小化とネットワーク境界を必ず確認します。

ID	試験	期待結果
A-01	社内LAN/VPNから内部URLへ接続	正式認証後にスケジュール表示
A-02	インターネットから内部URLへ接続	FWまたはNginxで拒否
A-03	予定の追加・編集・削除・複数操作・undo	SQLiteへ保存、version増加、履歴記録
A-04	機器利用・キャンセル待ち・機器点検を各時間帯で登録	外部の▲/▼/予約済み/ーが予定表と一致
A-05	署名なし・誤署名・6分前timestampでPOST	401で拒否しJSONを変更しない
A-06	不正room、状態、日付、128KiB超をPOST	400または413で拒否
A-07	古いsourceVersionをPOST	409で拒否し新しいJSONを保持
A-08	外部サーバー通信を遮断して内部予定を保存	内部保存は成功、pending=1、画面へ連携失敗通知
A-09	通信復旧後にtimer実行	最新世代だけ送信しpending=0
A-10	公開API本文を検索	利用者名・件名・メモ・履歴・内線が存在しない
A-11	外部URLの未知パス、api.php、SQLite候補へ接続	すべて404/403。ディレクトリー覧なし
A-12	月替わりを検証用時刻またはfixtureで再現	当月を含む3か月だけ。過去月ファイルなし
A-13	同時保存とJSON閲覧を連続実行	部分JSON・500・破損なし
A-14	バックアップから検証環境へ復元	integrity_check成功、予定件数一致

公開JSONの確認例

```
curl -fsS https://<PUBLIC_FQDN>/public-availability.php | jq .
curl -fsS https://<PUBLIC_FQDN>/public-availability.php \
| jq '{schemaVersion,sourceVersion,updatedAt,rangeStart,rangeEnd,rooms:{.availability|keys}}'
```

十四 監視・バックアップ・保守

内部業務データと公開派生データで、保護水準と復旧方法を分けます。

対象	監視 / バックアップ	基準
内部SQLite	毎日sqlite3 .backup、暗号化して別媒体へ。定期復元試験	RPO/RTOは組織で決定。最低7日世代を例示
公開JSON	updatedAtの鮮度、HTTP 200、schemaVersionを監視	再生成可能。長期バックアップ不要
送信	service終了コード、連続失敗、pending、最後の成功	15分超失敗で警告等を運用決定
Web	内部/外部HTTP、TLS期限、PHP-FPM、Nginx 5xx	外部は別拠点からも監視
ディスク	DB領域、ログ、リリース領域	80%警告、90%緊急の例
時刻	chrony同期状態、offset	300秒より十分小さい閾値
セキュリティ	受信401/409急増、SSH失敗、変更監査	通常値からの逸脱を通知

SQLiteバックアップ例

```
BACKUP=/var/backups/kptc-scheduler/group-watcher-$(date +%F-%H%M).sqlite
sudo -u kptc sqlite3 /var/lib/kptc-scheduler/group-watcher.sqlite \
  ".backup '$BACKUP'"
sqlite3 "$BACKUP" 'PRAGMA integrity_check;'
sha256sum "$BACKUP" > "$BACKUP.sha256"
```

鍵・証明書の保守

- HMAC鍵は年1回または漏えい疑い時にローテーションします。移行期間はcurrent/nextの2鍵検証を実装すると無停止で交換できます。
- TLS証明書は自動更新し、更新失敗と残存日数を監視します。
- 退職者・委託終了者のSSH鍵、認証権限、パスワード管理庫アクセスを速やかに削除します。

十五 障害復旧・ロールバック

内部保存、外部表示、連携のどこが故障したかを分けて判断します。

事象	初動	復旧
外部へ送れない	内部予定は継続。pendingとserviceログ、DNS/TLS/時刻を確認	原因修正後にCLI手動実行。JSON本文をメール等で送らない
外部JSON破損	受信APIを一時停止し、直前の正常表示を確認	内部から再生成・再送。公開DB復元は不要
内部DB破損	書込停止、ファイル保全、WALを含めて退避	最新正常backupを検証環境へ復元後、version確認して切替
新リリース不具合	current symlinkを直前リリースへ戻す	Nginx/PHP-FPM reload、DB migrationの後方互換性を確認
鍵漏えい	受信APIを閉じ、鍵を無効化、アクセスログ保全	新鍵配布、2台再起動、正常JSONを再送
内部侵害疑い	外部送信と内部アクセスを遮断、証跡保全	組織のインシデント手順に従い再構築。外部JSONも再発行

アプリリリースの戻し方

```
# 内部サーバー例。DBを巻き戻す前にアプリだけを戻す
sudo ln -sfn /opt/kptc/releases/<PREVIOUS>/internal /var/www/kptc-internal/current
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl reload php*-fpm

# 外部サーバーも同様にpublicリリースのsymlinkを戻す
```

- DB移行を伴う更新では、コードのロールバックだけで旧版がDBを読めるよう後方互換を確保します。
- 公開JSONは派生データです。過去月を復元せず、内部DBから現行3か月を再生成します。
- 障害時の手動JSON編集は禁止します。表示内容と内部予定の不一致が残るためです。

十六 実装・運用チェックリスト

未確定項目を埋め、すべての必須項目が完了してから本番切替を承認します。

No.	確認項目	状態
01	内部FQDN、外部FQDN、内部CIDR、管理CIDR、固定送信元IPを確定	☐
02	正式認証方式、ユーザー同期、権限、セッション期限を確定	☐
03	内部→外部のHTTPS送信、HMAC、リプレイ防止を実装・レビュー	☐
04	受信JSONのスキーマ・世代・サイズ・許可値検証を実装	☐
05	内部用・外部用のホワイトリスト配布物を分離	☐
06	外部配布物に内部画面、API、SQLite、個人情報がないことを確認	☐
07	内部SQLiteの環境変数対応と権限・WAL・backupを確認	☐
08	Nginx、PHP-FPM、FW、TLS、時刻同期、SSH制限を確認	☐
09	保存直後送信、5分再送、月替わり、pending監視を確認	☐
10	受入試験A-01～A-14を実施し証跡を保存	☐
11	切替・ロールバック責任者、連絡網、作業時間帯を確定	☐
12	さくら環境の停止日、保管期限、削除承認を確定	☐

環境別パラメータ記入欄

項目	値	項目	値
内部FQDN / 内部CIDR		外部FQDN / 固定送信元IP	
OS / PHP / Nginx		認証方式 / IdP	
証明書 / 更新方式		監視先 / 通知先	
バックアップ / RPO / RTO		本番切替日 / 戻し判断時刻	

文書管理・次工程

本手順書は構築方針と作業順序を定めます。環境値の確定後、実装変更を別作業として行います。

管理項目	内容
文書名	KPTC Scheduler 独立Linuxサーバー構築・移行手順書
版 / 基準日	1.0 / 2026-08-15
現行実装基準	GitHub main / abe28ac
対象構成	内部スケジューラー1台 + 外部空き状況1台。公開DBなし、3か月JSONのみ
推奨連携	内部から外部へのHTTPS POST + HMAC-SHA256 + 再送timer
未実装事項	遠隔送受信API、配布物分離、正式認証、運用監視は今後の実装対象

次工程

工程	成果物
1. 基本設計確定	FQDN、CIDR、OS、認証、証明書、監視、RPO/RTOの決定票
2. アプリ改修	送信・受信・配布物分離・DB/バス環境変数・正式認証
3. 検証環境構築	内部/外部2台、テスト証明書、HMAC鍵、受入試験証跡
4. 本番構築	サーバー設定、バックアップ、監視、運用手順、復旧訓練
5. 移行	SQLite整合backup、切替記録、監視記録、さくら停止計画

最優先 正式認証とネットワーク境界を確定する。	実装 外向き送信と受信APIを追加し、配布物を分離する。	運用 再送・監視・バックアップ・復旧試験まで含めて承認する。
---	--	--